

Part III: Some Optimization Theory

RTG Summer School

Yimin Zhong (yimin.zhong@auburn.edu)

Convergence Theory

The Central Question

How quickly does an algorithm converge to the optimal solution x^* ?

In large-scale machine learning, the **non-asymptotic** rate is paramount.

We need to know exactly (as much as possible) how many iterations k are required to reach an ϵ -neighborhood of the minimum.

Definition (L -smoothness)

f is L -smooth if for all x, y :

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

Descent Lemma:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2$$

Definition (μ -strong convexity)

f is μ -strongly convex if for all x, y :

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{\mu}{2} \|y - x\|^2$$

- Function is at least as curved as a quadratic
- Guarantees unique global minimum
- Prevents landscape from becoming too flat

Theorem (Co-Coercivity)

If f is convex and ∇f is L -Lipschitz, then:

$$\langle \nabla f(x) - \nabla f(y), x - y \rangle \geq \frac{1}{L} \|\nabla f(x) - \nabla f(y)\|^2$$

Key consequence: Set $y = x^*$ (global minimizer, $\nabla f(x^*) = 0$):

$$\langle \nabla f(x), x - x^* \rangle \geq \frac{1}{L} \|\nabla f(x)\|^2$$

This lower-bounds the inner product $\langle \nabla f(x), x - x^* \rangle$ purely in terms of the gradient norm. It's the key that makes the entire GD proof work.

Gradient Descent Convergence — Convex Case

GD: $x_{k+1} = x_k - \eta \nabla f(x_k)$

Theorem (GD for L -smooth convex)

With $\eta = 1/L$:

$$f(x_k) - f(x^*) \leq \frac{L \|x_0 - x^*\|^2}{2k}$$

Key: GD achieves $O(1/k)$ convergence.

Question

Is this the best possible rate? Can we do better?

Proof Step 1: Progress Per Iteration

Start from the **Descent Lemma** (which follows from L -smoothness):

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2$$

Plug in the GD update: $x = x_k, y = x_{k+1} = x_k - \eta \nabla f(x_k)$.

With $\eta = \frac{1}{L}$:

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) + \langle \nabla f(x_k), -\frac{1}{L} \nabla f(x_k) \rangle + \frac{L}{2} \left\| \frac{1}{L} \nabla f(x_k) \right\|^2 \\ &= f(x_k) - \frac{1}{L} \|\nabla f(x_k)\|^2 + \frac{1}{2L} \|\nabla f(x_k)\|^2 = f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2 \end{aligned}$$

Key inequality

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2$$

Proof Step 2: GD Does Not Move Away from x^*

Expand the squared distance:

$$\begin{aligned}\|x_{k+1} - x^*\|^2 &= \|x_k - \eta \nabla f(x_k) - x^*\|^2 \\ &= \|x_k - x^*\|^2 - 2\eta \langle \nabla f(x_k), x_k - x^* \rangle + \eta^2 \|\nabla f(x_k)\|^2\end{aligned}$$

Apply **co-coercivity** with $y = x^*$ (where $\nabla f(x^*) = 0$):

$$\langle \nabla f(x_k), x_k - x^* \rangle \geq \frac{1}{L} \|\nabla f(x_k)\|^2$$

Substitute:

$$\begin{aligned}\|x_{k+1} - x^*\|^2 &\leq \|x_k - x^*\|^2 - \frac{2\eta}{L} \|\nabla f(x_k)\|^2 + \eta^2 \|\nabla f(x_k)\|^2 \\ &= \|x_k - x^*\|^2 + \eta \left(\eta - \frac{2}{L} \right) \|\nabla f(x_k)\|^2\end{aligned}$$

With $\eta \leq \frac{2}{L}$, the second term is ≤ 0 . Hence $\|x_{k+1} - x^*\| \leq \|x_k - x^*\| \leq \dots \leq \|x_0 - x^*\| =: R$. ■

Proof Step 3: Relate Gradient to Suboptimality

Let $\delta_k := f(x_k) - f(x^*)$.

Convexity gives:

$$\begin{aligned} f(x^*) &\geq f(x_k) + \langle \nabla f(x_k), x^* - x_k \rangle \\ \implies \delta_k = f(x_k) - f(x^*) &\leq \langle \nabla f(x_k), x_k - x^* \rangle \end{aligned}$$

Apply **Cauchy-Schwarz** + Step 2 ($\|x_k - x^*\| \leq R$):

$$\delta_k \leq \|\nabla f(x_k)\| \cdot \|x_k - x^*\| \leq \|\nabla f(x_k)\| \cdot R$$

$$\|\nabla f(x_k)\| \geq \frac{\delta_k}{R} \quad \implies \quad \|\nabla f(x_k)\|^2 \geq \frac{\delta_k^2}{R^2}$$

Proof Step 4: The Recurrence

Combine Step 1 and Step 3. Subtract $f(x^*)$ from the Step 1 inequality:

$$\delta_{k+1} \leq \delta_k - \frac{1}{2L} \|\nabla f(x_k)\|^2$$

Substitute the lower bound $\|\nabla f(x_k)\|^2 \geq \delta_k^2/R^2$:

$$\delta_{k+1} \leq \delta_k - \frac{\delta_k^2}{2LR^2}$$

The recurrence relation

$$\delta_{k+1} \leq \delta_k \left(1 - \frac{\delta_k}{2LR^2} \right)$$

This says: progress $\delta_k - \delta_{k+1} \geq \frac{\delta_k^2}{2LR^2}$ is **quadratic** in the current error. Slow when δ_k is small!

Proof Step 5: Solve the Recurrence

From $\delta_{k+1} \leq \delta_k - \frac{\delta_k^2}{2LR^2}$:

$$\delta_k - \delta_{k+1} \geq \frac{\delta_k^2}{2LR^2}$$

Divide both sides by $\delta_k \delta_{k+1}$ (both positive):

$$\frac{1}{\delta_{k+1}} - \frac{1}{\delta_k} = \frac{\delta_k - \delta_{k+1}}{\delta_k \delta_{k+1}} \geq \frac{\delta_k^2}{2LR^2 \delta_k \delta_{k+1}} = \frac{\delta_k}{2LR^2 \delta_{k+1}} \geq \frac{1}{2LR^2}$$

(the last step uses $\delta_k \geq \delta_{k+1}$ from the descent guarantee)

Telescoping sum from $i = 0$ to $k - 1$:

$$\frac{1}{\delta_k} - \frac{1}{\delta_0} \geq \frac{k}{2LR^2} \implies \frac{1}{\delta_k} \geq \frac{k}{2LR^2} \implies \delta_k \leq \frac{2LR^2}{k}$$

This is exactly $f(x_k) - f(x^*) \leq \frac{2L\|x_0 - x^*\|^2}{k}$. ■

Gradient Descent — Strongly Convex Case

Theorem (GD for μ -strongly convex, L -smooth)

With $\eta = 1/L$:

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \left(1 - \frac{\mu}{L}\right)^k (f(\mathbf{x}_0) - f(\mathbf{x}^*))$$

Linear convergence with rate $\rho = 1 - \frac{\mu}{L}$.

Condition number $\kappa = L/\mu$ determines speed:

- $\kappa = 1$: convergence in 1 step (the function is exactly quadratic)
- $\kappa = 100$: $\rho = 0.99$, need ≈ 1833 iterations for $\epsilon = 10^{-6}$.

Lower Bounds for First-Order Methods

A first-order method chooses x_{k+1} from:

$$x_0 + \text{span}\{\nabla f(x_0), \dots, \nabla f(x_k)\}$$

Theorem (Nesterov's Lower Bound)

For any $k < \frac{1}{2}(n - 1)$, there exists an L -smooth convex f such that:

$$f(x_k) - f(x^*) \geq \frac{3L\|x_0 - x^*\|^2}{32(k+1)^2}$$

No first-order method can beat $O(1/k^2)$ on general convex functions — GD at $O(1/k)$ is **suboptimal!**

Lower Bound: Proof Construction

Worst-case function: $f(x) = \frac{L}{8}(x^T A x - 2x_1)$ with A tridiagonal:

$$A = \begin{pmatrix} 2 & -1 & 0 & \cdots \\ -1 & 2 & -1 & \cdots \\ 0 & -1 & 2 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}, \quad \nabla f(x) = \frac{L}{4}(Ax - e_1)$$

Starting point: $x_0 = \mathbf{0} \in \mathbb{R}^n$.

Step $k = 1$: $\nabla f(x_0) = -\frac{L}{4}e_1$. Any first-order method picks $x_1 \in x_0 + \text{span}\{\nabla f(x_0)\} = \text{span}\{e_1\}$, so x_1 has only a first coordinate.

Step $k = 2$: Ax_1 involves e_2 (since A is tridiagonal), so $\nabla f(x_1) \in \text{span}\{e_1, e_2\}$. Thus $x_2 \in \text{span}\{e_1, e_2\}$.

By induction: After k iterations, $x_k \in \text{span}\{e_1, \dots, e_k\}$ — the method only “sees” the first k coordinates. The true minimum x^* uses all n coordinates, so the gap is unavoidable.

Polyak's Heavy-Ball Method

Polyak (1964) proposed adding momentum to reduce oscillations of GD:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1})$$

Theorem (Polyak, convex + L -smooth)

Set $\beta = 1 - \sqrt{\eta L}$. If $\eta \leq 1/L$, then:

$$\min_{0 \leq k \leq T} f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \frac{2\sqrt{\eta L} - \eta L}{2\eta(T+1)} \|\mathbf{x}_0 - \mathbf{x}^*\|^2$$

Compare: GD achieves $\frac{1}{2\eta(T+1)} \|\mathbf{x}_0 - \mathbf{x}^*\|^2$. Factor $2\sqrt{\eta L} - \eta L \leq 1$, strict when $\eta < 1/L$. Equal at $\eta = 1/L$.

Key proof idea: Define $\mathbf{p}_k = \frac{\beta}{1-\beta}(\mathbf{x}_k - \mathbf{x}_{k-1})$. Then $\mathbf{x}_{k+1} + \mathbf{p}_{k+1} = \mathbf{x}_k + \mathbf{p}_k - \frac{\eta}{1-\beta} \nabla f(\mathbf{x}_k) -$ exactly the GD structure!

Nesterov Accelerated Gradient (NAG)

Heavy-Ball: $x_{k+1} = \gamma_k - \eta \nabla f(x_k), \quad \gamma_k = x_k + \beta(x_k - x_{k-1})$

NAG: $x_{k+1} = \gamma_k - \eta \nabla f(\gamma_k), \quad \gamma_k = x_k + \beta(x_k - x_{k-1})$

NAG evaluates the gradient at the **lookahead** point γ_k — if momentum helps, why not use it in the gradient computation? This small change makes the convergence rate jump from $O(1/T)$ to $O(1/T^2)$.

NAG: Convergence Rate

NAG-c algorithm: $\eta \leq 1/L$, $\beta_k = \frac{k}{k+2}$.

$$\mathbf{x}_{k+1} = \gamma_k - \eta \nabla f(\gamma_k), \quad \gamma_k = \mathbf{x}_k + \beta_k(\mathbf{x}_k - \mathbf{x}_{k-1})$$

Theorem (Convergence of NAG-c)

If f is L -smooth and convex, then:

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}^*) \leq \frac{2}{\eta(k+2)^2} \|\mathbf{x}_0 - \mathbf{x}^*\|^2$$

NAG achieves $O(1/k^2)$ — precisely matches Nesterov's lower bound! NAG is an **optimal** first-order method for convex optimization.

NAG Proof, Step 1: A Key Inequality

For any $\mathbf{x}$, define $\mathbf{x}_+ := \mathbf{x} - \eta \nabla f(\mathbf{x})$. From the descent lemma with $\eta \leq 1/L$:

$$f(\mathbf{x}_+) - f(\mathbf{x}) \leq -\frac{1}{2\eta} \|\mathbf{x}_+ - \mathbf{x}\|^2$$

For any $\mathbf{z}$, using convexity $f(\mathbf{z}) \geq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{z} - \mathbf{x} \rangle$:

$$\begin{aligned} f(\mathbf{x}_+) - f(\mathbf{z}) &\leq f(\mathbf{x}) - f(\mathbf{z}) - \frac{\eta}{2} \|\nabla f(\mathbf{x})\|^2 \\ &\leq \langle \nabla f(\mathbf{x}), \mathbf{x} - \mathbf{z} \rangle - \frac{\eta}{2} \|\nabla f(\mathbf{x})\|^2 \\ &= -\frac{1}{2\eta} \|\mathbf{x}_+ - \mathbf{z}\|^2 + \frac{1}{2\eta} \|\mathbf{x} - \mathbf{z}\|^2 \end{aligned}$$

$$f(\mathbf{x}_+) - f(\mathbf{z}) \leq \frac{1}{2\eta} (\|\mathbf{x} - \mathbf{z}\|^2 - \|\mathbf{x}_+ - \mathbf{z}\|^2)$$

NAG Proof, Step 2: Weighted Combination

NAG-c recall: $\gamma_k = \theta_k + \beta_k(\theta_k - \theta_{k-1})$, $\theta_{k+1} = \gamma_k - \eta \nabla f(\gamma_k)$.

Apply Step 1 with $x = \gamma_k$:

- $z = \theta^*$ (global min): $f(\theta_{k+1}) - f(\theta^*) \leq \frac{1}{2\eta}(\|\gamma_k - \theta^*\|^2 - \|\theta_{k+1} - \theta^*\|^2)$
- $z = \theta_k$: $f(\theta_{k+1}) - f(\theta_k) \leq \frac{1}{2\eta}(\|\gamma_k - \theta_k\|^2 - \|\theta_{k+1} - \theta_k\|^2)$

Combine as $(1 - \beta_k) \times (\text{first}) + \beta_k \times (\text{second})$. The RHS simplifies via **affinity**: for any z , $(1 - \beta_k)\|\gamma_k - z\|^2 + \beta_k\|\gamma_k - \theta_k\|^2 = (1 - \beta_k)\|\gamma_k - z_\beta\|^2$ where $z_\beta = (1 - \beta_k)\theta^* + \beta_k\theta_k$, plus an extra $\|\theta_k - \theta^*\|^2$ term from the θ_k part.

After simplification:

$$\begin{aligned} f(\theta_{k+1}) - ((1 - \beta_k)f(\theta^*) + \beta_k f(\theta_k)) \leq \\ \frac{(1 - \beta_k)^2}{2\eta} \left(\left\| \theta_k + \frac{\beta_k}{1 - \beta_k}(\theta_k - \theta_{k-1}) - \theta^* \right\|^2 \right. \\ \left. - \left\| \theta_{k+1} + \frac{\beta_k}{1 - \beta_k}(\theta_{k+1} - \theta_k) - \theta^* \right\|^2 \right) \end{aligned}$$

NAG Proof, Step 3: Choosing β_k

The inequality from Step 2 has the form $A_{k+1} + B_{k+1} \leq \frac{\beta_k}{(1-\beta_k)^2} A_k + B_k$ where $A_k \propto f(\theta_k) - f(\theta^*)$ and B_k is a squared distance term.

To telescope, we need $\frac{\beta_k}{(1-\beta_k)^2} \leq \frac{1}{(1-\beta_{k-1})^2}$.

Choose $\beta_k = \frac{k}{k+2}$. Then:

$$\begin{aligned}\frac{\beta_k}{(1-\beta_k)^2} &= \frac{k/(k+2)}{(2/(k+2))^2} = \frac{k(k+2)}{4} \\ \frac{1}{(1-\beta_{k-1})^2} &= \frac{1}{(2/(k+1))^2} = \frac{(k+1)^2}{4}\end{aligned}$$

Since $k(k+2) = k^2 + 2k \leq k^2 + 2k + 1 = (k+1)^2$, the condition holds.

NAG Proof, Step 4: Closing the Bound

With $\beta_k = \frac{k}{k+2}$, the monotone property iterates back to $k = 0$:

$$\frac{\eta}{(1 - \beta_k)^2} (f(\theta_{k+1}) - f(\theta^*)) \leq \frac{\eta}{(1 - \beta_0)^2} (f(\theta_{-1}) - f(\theta^*)) + \frac{1}{2} \|\theta_0 - \theta^*\|^2$$

Set $\theta_{-1} = \theta_0$, so $f(\theta_{-1}) = f(\theta_0)$. Using L -smoothness:

$$f(\theta_0) - f(\theta^*) \leq \frac{L}{2} \|\theta_0 - \theta^*\|^2$$

With $1 - \beta_0 = 1$ and $1 - \beta_k = \frac{2}{k+2}$, and $\eta \leq 1/L$:

$$\frac{\eta(k+2)^2}{4} (f(\theta_{k+1}) - f(\theta^*)) \leq \frac{\eta L}{2} \|\theta_0 - \theta^*\|^2 + \frac{1}{2} \|\theta_0 - \theta^*\|^2 \leq \|\theta_0 - \theta^*\|^2$$

$$\implies f(\theta_{k+1}) - f(\theta^*) \leq \frac{2}{\eta(k+2)^2} \|\theta_0 - \theta^*\|^2 = O(1/k^2) \quad \blacksquare$$

NAG for Strongly Convex Functions

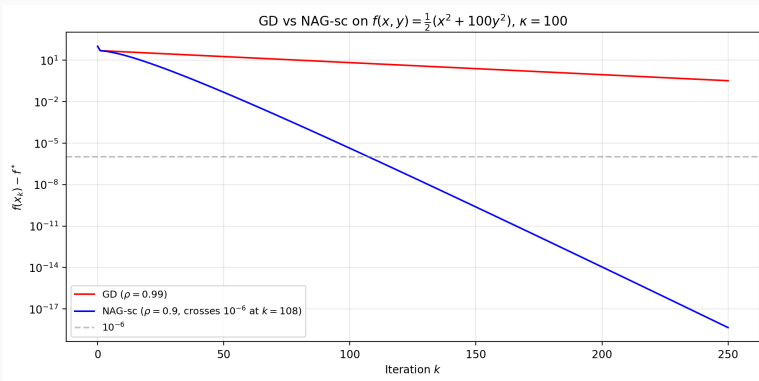
NAG-sc: $\eta = \frac{1}{L}, \beta = \frac{1-\sqrt{\kappa}}{1+\sqrt{\kappa}}$ where $\kappa = \frac{\mu}{L}$.

Theorem

If f is μ -strongly convex and L -smooth:

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq (1 - \sqrt{\kappa})^k \left(f(\mathbf{x}_0) - f(\mathbf{x}^*) + \frac{\mu}{2} \|\mathbf{x}_0 - \mathbf{x}^*\|^2 \right)$$

Simulation: GD vs NAG-sc on $f(x, y) = \frac{1}{2}(x^2 + 100y^2)$



$\kappa = 100$, $x_0 = (10, 1)$, $\eta = 1/100$. GD crosses 10^{-6} at $k = 882$; NAG-sc crosses at $k = 108$ — **$8 \times$ speedup**.
Theoretical bounds: $\text{GD} \leq \ln(10^{-8}) / \ln(0.99) \approx 1833$, $\text{NAG} \leq \ln(10^{-8}) / \ln(0.9) \approx 175$.

Remark: NAG as a Second-Order ODE

NAG corresponds to the ODE (Su, Boyd, Candès 2016):

$$\ddot{x} + \frac{3}{t}\dot{x} + \nabla f(x) = 0$$

Theorem (Nesterov ODE Convergence)

For $\ddot{x} + \frac{a}{t}\dot{x} + b\nabla f(x) = 0$ with $a \geq 3$:

$$f(x(T)) - f^* \leq \frac{(a-1)^2}{2b} \frac{\|x_0 - x^*\|^2}{T^2}$$

Proof sketch: Use Lyapunov $V = t^2(f(x) - f^*) + \frac{1}{2b}\|(a-1)(x - x^*) + t\dot{x}\|^2$. Then $\frac{dV}{dt} \leq t(3-a)(f - f^*) \leq 0$ when $a \geq 3$. For $a = 3$, the constant matches NAG-c: $\frac{2}{bT^2}$.

Definition (Polyak-Łojasiewicz Inequality)

f satisfies the PL inequality with constant $\mu > 0$ if:

$$\frac{1}{2} \|\nabla f(\mathbf{x})\|^2 \geq \mu(f(\mathbf{x}) - f(\mathbf{x}^*))$$

- Every μ -strongly convex function is PL
- **Not vice versa:** $f(x) = x^2 + 3 \sin^2(x)$ is PL but non-convex
- PL functions have no bad local minima — every stationary point is global

Theorem (GD for PL Functions)

If f is L -smooth and μ -PL, GD with $\eta = 1/L$ achieves:

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \left(1 - \frac{\mu}{L}\right)^k (f(\mathbf{x}_0) - f(\mathbf{x}^*))$$

Proof: Descent lemma + PL inequality directly yields the linear rate. No convexity needed! ■

Significance: Explains why gradient descent works on many neural network loss landscapes.

Stochastic Algorithms

Stochastic Optimization

Goal: Learn a map $f(\cdot, w)$ from data $\{(x_i, y_i)\}_{i=1}^n \sim \mathcal{P}$.

The **risk** (or expected loss) measures how well w performs on unseen data:

$$R(w) = \mathbb{E}_{(x,y) \sim \mathcal{P}} [\ell(y, f(x, w))]$$

But $\mathcal{P}$ is unknown — we only have a finite sample. The **empirical risk** is the sample-based proxy:

$$\hat{R}_n(w) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i, w))$$

Training minimizes $\hat{R}_n$, hoping it approximates R . The gradient $\nabla \hat{R}_n$ costs $O(n)$ per step, which may be intractable for large n .

SGD: Replace $\nabla \hat{R}_n$ with an **unbiased minibatch estimate** $\nabla \ell(y_i, f(x_i, w))$. Noise is the price of scalability.

SGD Convergence for Strongly Convex Functions

Theorem (SGD with decaying step size)

If $\mathcal{L}$ is L -smooth, μ -strongly convex, and $\mathbb{E}[\|g\|^2] \leq G^2$, then with $\eta_k = \frac{1}{\mu k}$:

$$\mathbb{E}[\|w_k - w^*\|^2] \leq \frac{\max\{\|w_0 - w^*\|^2, G^2/\mu^2\}}{k}$$

SGD achieves $O(1/k)$, it is much slower than GD's linear rate e^{-k} . The **non-vanishing variance** is the issue.

SGD Proof Sketch: Set Up

Let $\delta_k = \mathbb{E}[\|w_k - w^*\|^2]$. The SGD step is $w_{k+1} = w_k - \eta_k g_k$ where g_k is an unbiased minibatch gradient.

Expand the squared distance:

$$\begin{aligned}\|w_{k+1} - w^*\|^2 &= \|w_k - \eta_k g_k - w^*\|^2 \\ &= \|w_k - w^*\|^2 - 2\eta_k \langle g_k, w_k - w^* \rangle + \eta_k^2 \|g_k\|^2\end{aligned}$$

Take expectations. $\mathbb{E}[g_k] = \nabla \mathcal{L}(w_k)$, and use the assumed bounded variance $\mathbb{E}[\|g_k\|^2] \leq G^2$.

Strong convexity gives $\langle \nabla \mathcal{L}(w_k), w_k - w^* \rangle \geq \mu \|w_k - w^*\|^2$:

$$\delta_{k+1} \leq (1 - 2\eta_k \mu) \delta_k + \eta_k^2 G^2$$

This recurrence looks similar to the GD case, but with an extra $\eta_k^2 G^2$ term from gradient noise.

SGD Proof Sketch: Solve the Recurrence

Set $\eta_k = 1/(\mu k)$. Then:

$$\delta_{k+1} \leq (1 - 2/k)\delta_k + G^2/(\mu^2 k^2)$$

One can verify by induction that $\delta_k \leq Q/k$ for $Q = \max\{\delta_0, G^2/\mu^2\}$:

$$\begin{aligned}\delta_{k+1} &\leq (1 - 2/k)\frac{Q}{k} + \frac{G^2}{\mu^2 k^2} = \frac{Qk - 2Q + G^2/\mu^2}{k^2} \\ &\leq \frac{Q}{k+1} \quad (\text{by choice of } Q)\end{aligned}$$

Key difference from GD: The $\eta_k^2 G^2$ noise term is **non-vanishing**, even as $w_k \rightarrow w^*$, gradient noise persists. GD's corresponding term $\eta^2 \|\nabla f\|^2$ vanishes as $\nabla f \rightarrow 0$, enabling linear convergence. SGD's persistent noise caps it at $O(1/k)$.

Variance Reduction: The SVRG Algorithm

Idea: Periodically compute a full gradient to anchor stochastic estimates.

SVRG (per epoch s)

1. Compute full gradient: $\mu = \nabla \mathcal{L}(\tilde{w}_s)$
2. For $t = 0, \dots, m - 1$:
 - Pick random i , compute:
 - $v_t = \nabla L(w_t, x_i) - \nabla L(\tilde{w}_s, x_i) + \mu$
 - Update $w_{t+1} = w_t - \eta v_t$

$\mathbb{E}[v_t] = \nabla \mathcal{L}(w_t)$ (unbiased), and as all points converge, variance $\rightarrow 0$.

Theorem (Linear Convergence of SVRG)

For L -smooth, μ -strongly convex $\mathcal{L}$, with suitable η and m :

$$\mathbb{E}[\mathcal{L}(\tilde{\mathbf{w}}_{s+1}) - \mathcal{L}(\mathbf{w}^*)] \leq \gamma \mathbb{E}[\mathcal{L}(\tilde{\mathbf{w}}_s) - \mathcal{L}(\mathbf{w}^*)]$$

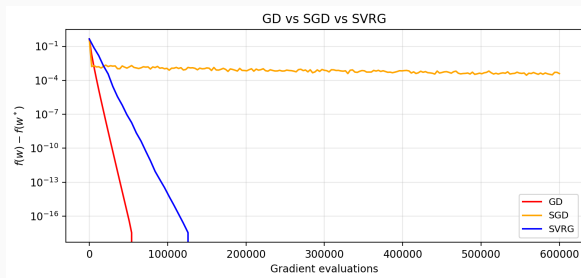
where $\gamma < 1$.

Key insights:

1. Conditioned on $\mathbf{w}_{t-1}$, the variance $\mathbb{E}[\|\mathbf{v}_t\|^2] \leq 4L(\mathcal{L}(\mathbf{w}_t) - \mathcal{L}(\mathbf{w}^*) + \mathcal{L}(\tilde{\mathbf{w}}_s) - \mathcal{L}(\mathbf{w}^*))$ (this vanishes as optimization progresses).
2. Conditioned on $\mathbf{w}_{t-1}$, estimate $\mathbb{E}[\|\mathbf{w}_t - \mathbf{w}^*\|^2]$.
3. Focus on one epoch and sum up.

Simulation: GD vs SGD vs SVRG

Least squares: $f(w) = \frac{1}{2n} \|Xw - y\|^2$, $n = 3000$, $d = 50$.



Method	Work / Epoch	Time / Epoch	Convergence
GD	1 full gradient	$\sim 100 \mu\text{s}$	Linear ($\rho \approx 0.99$)
SGD	n single-sample steps	$\sim 16 \text{ ms}$	Sublinear (noisy)
SVRG	1 full + n inner steps	$\sim 47 \text{ ms}$	Linear (matches GD)

Adaptive & Second-Order Methods

Beyond Isotropic Scaling

Limitation of GD: Treated every gradient direction equally. Training is slowed down by certain directions.

Example: $f(x, y) = \frac{1}{2}(x^2 + 100y^2)$. GD cannot choose a larger learning rate in the y direction.

Hessian in deep learning is highly ill-conditioned:

- Spectral bulk of small eigenvalues (low rank structure)
- A few outlier eigenvalues (very large)

Solution: Adaptively choose learning rates. Rescale the gradient differently in each direction so that steep directions get small steps and flat directions get large steps.

Second-order Taylor expansion:

$$f(\mathbf{w} + \delta\mathbf{w}) \approx f(\mathbf{w}) + \nabla f(\mathbf{w})^T \delta\mathbf{w} + \frac{1}{2} \delta\mathbf{w}^T H(\mathbf{w}) \delta\mathbf{w}$$

Minimizing the quadratic approximation w.r.t. $\delta\mathbf{w}$:

$$\frac{\partial}{\partial \delta\mathbf{w}} \left(\nabla f(\mathbf{w})^T \delta\mathbf{w} + \frac{1}{2} \delta\mathbf{w}^T H(\mathbf{w}) \delta\mathbf{w} \right) = \nabla f(\mathbf{w}) + H(\mathbf{w}) \delta\mathbf{w} = \mathbf{0}$$

Newton step:

$$\delta\mathbf{w} = -H(\mathbf{w}_k)^{-1} \nabla f(\mathbf{w}_k), \quad \mathbf{w}_{k+1} = \mathbf{w}_k + \delta\mathbf{w}$$

Intuitively: pre-multiply the gradient by the inverse Hessian to rescale each direction by its curvature.

Newton's Limitations in Deep Learning

1. **Computational Cost:** H^{-1} costs $O(d^3)$ — impossible for $d \approx 10^8$
2. **Saddle Point Attraction:** Seeks points where $\nabla = 0$, regardless of curvature — attracted to saddles
3. **Stochasticity:** Hessian is extremely noisy on minibatches

Need approximations that preserve curvature information at a tractable cost.

Quasi-Newton: The Secant Condition

Core idea: Newton needs $H(w_k)$ (too expensive). Instead, *approximate* $B_k \approx H(w_k)$ from gradient changes, which are cheap.

Taylor's theorem gives the **secant equation**:

$$\nabla f(w_{k+1}) \approx \nabla f(w_k) + H(w_{k+1})(w_{k+1} - w_k)$$

Define the step and gradient difference:

$$s_k = w_{k+1} - w_k, \quad y_k = \nabla f(w_{k+1}) - \nabla f(w_k)$$

The secant equation becomes $y_k \approx H(w_{k+1})s_k$.

The secant condition: B_{k+1} satisfies:

$$B_{k+1}s_k = y_k$$

This is one underdetermined linear equation, and has infinitely many solutions. We need a rule to pick one.

BFGS: The Update Formula

Among all matrices satisfying $B_{k+1}s_k = y_k$, **BFGS** picks the one **closest** to B_k (in Frobenius norm):

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

Rank-2 correction:

- First term: along the $B_k s_k$ direction
- Second term: along y_k direction

If $y_k^T s_k > 0$ (guaranteed by Wolfe line search conditions), B_{k+1} stays positive definite.

Superlinear Convergence: $\lim_{k \rightarrow \infty} \frac{\|w_{k+1} - w^*\|}{\|w_k - w^*\|} = 0$.

L-BFGS: Quasi-Newton at Scale

BFGS stores $B_k \in \mathbb{R}^{d \times d}$ ($O(d^2)$ memory), expensive for $d > 10^5$.

L-BFGS (Nocedal, 1980): Store only the last m vector pairs $\{(s_i, y_i)\}_{i=k-m}^{k-1}$ where $m \sim 5$ to 20.

Two-loop recursion: The product $B_k^{-1} \nabla f(w_k)$ can be computed *implicitly* from just these $2m$ vectors, without ever forming B_k :

1. Forward pass: compute α_i coefficients from $\{s_i, y_i\}$ and $\nabla f(w_k)$
2. Backward pass: assemble the preconditioned direction from $\{\alpha_i, s_i, y_i\}$

Cost: $O(md)$ memory and $O(md)$ flops per iteration.

Convergence: Superlinear in practice. The theory yields a linear rate, but for $m \geq 20$, the practical behavior is nearly indistinguishable from that of full BFGS.

All second-order methods need to compute an approximation for the Hessian.

Theorem (Bartlett Identity)

Let $p(y \mid x; w)$ be a smooth probability model and $f(w) = -\log p(y \mid x; w)$ be the negative log-likelihood. Then at the true parameter w^* ,

$$\mathbb{E}_{y \sim p(\cdot \mid x; w^*)} [\nabla f(w^*) \cdot \nabla f(w^*)^\top] = \mathbb{E}_{y \sim p(\cdot \mid x; w^*)} [\nabla^2 f(w^*)]$$

The matrix $F = \mathbb{E}[\nabla \log p \cdot \nabla \log p^\top]$ is called the Fisher information matrix.

Quick Verification

1. $\nabla f = -\frac{\nabla p}{p}.$
2. $\nabla^2 f = -\frac{\nabla^2 p}{p} + \frac{\nabla p \cdot \nabla p^\top}{p^2} = -\frac{\nabla^2 p}{p} + \nabla f \cdot \nabla f^\top$
3. Take expectation

$$\begin{aligned}\mathbb{E}_{y \sim p}[\nabla^2 f] &= - \underbrace{\int \frac{\nabla^2 p}{p} p dy}_{\text{exchange integration and gradient}} + \mathbb{E}_{y \sim p}[\nabla f \cdot \nabla f^\top] \\ &= \mathbf{0} + \mathbb{E}_{y \sim p}[\nabla f \cdot \nabla f^\top].\end{aligned}$$

The diagonal entries of F are:

$$F_{ii} = \mathbb{E}[(\partial_i \log p)^2] = \mathbb{E}[(\partial_i f)^2] = \mathbb{E}[g_i^2].$$

This is the **second moment** of the i -th gradient component.

Estimate of Diagonal: No direct computation. Just track the running second moment of each gradient component:

$$\mathbf{v}_{t,i} = \beta_2 \mathbf{v}_{t-1,i} + (1 - \beta_2) g_{t,i}^2 \approx \mathbb{E}[g_i^2]$$

Then $\text{diag}(\sqrt{\mathbf{v}_t})^{-1}$ is a cheap, noisy proxy for the diagonal entries.

Adam as a Diagonal Approximation to $H^{-1/2}$

Diagonal preconditioning: Preconditioned with $(\text{diag}(F))^{-1/2} \approx H^{-1/2}$.

A steep direction has a large $|g_i|$, so v_i is large, so the step $g_i/\sqrt{v_i}$ is small. A flat direction has a small $|g_i|$, so v_i is small, so the step is large.

Cost: The full $H^{-1/2}$ is $O(d^3)$ to compute and $O(d^2)$ to store. The diagonal is $O(d)$ for both.

Other Variants: All adaptive optimizers make this same diagonal approximation, differing only in how they accumulate g^2 :

- **AdaGrad:** $\sum_{s=1}^t g_s^2$ — monotonic, learning rate decays forever
- **RMSProp:** EMA of g_t^2 — adapts to non-stationary curvature
- **Adam:** EMA + bias correction — the standard choice

Adam maintains two exponential moving averages per parameter:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{(first moment — momentum),} \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{(second moment — diagonal } H^{-1} \text{ proxy).}\end{aligned}$$

After bias correction (due to the leakage at initial time): $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$, then

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ are defaults. The bias correction is important early in training — without it, $m_1 = (1 - \beta_1)g_1$ (biased toward zero).

AdamW: Decoupled Weight Decay

The problem with Adam + L_2 : Original Adam adds λw to the gradient:

$$g_t \leftarrow g_t + \lambda w_{t-1}$$

The adaptive scaling then divides this by $\sqrt{\hat{v}_t}$, so parameters with small $\hat{v}_t$ experience **stronger** weight decay — the opposite of the intended uniform regularization.

AdamW (Loshchilov & Hutter, 2019) applies weight decay **directly** to the weights:

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Now, weight decay and adaptive gradient scaling are **decoupled** — every parameter gets the same λ regularization, regardless of its gradient history.

This improves generalization, and **AdamW** is now the default optimizer for most language models.

AdaGrad (Duchi et al., 2011): Accumulate the sum of squared gradients for each parameter $w^{(i)}$:

$$w_{t+1}^{(i)} = w_t^{(i)} - \frac{\eta}{\sqrt{\sum_{s=1}^t (g_s^{(i)})^2 + \varepsilon}} g_t^{(i)}$$

Parameters with historically large gradients get small steps. Parameters with small gradients get large steps.

The catch: The sum $\sum g^2$ grows *monotonically*, so the effective learning rate $\eta / \sqrt{\sum g^2}$ decays to zero. On non-convex problems (deep learning), this can stop learning prematurely.

From AdaGrad to Adam Family to Shampoo

The full matrix view: In full generality, AdaGrad preconditions by $(\sum_{s=1}^t g_s g_s^T)^{-1/2}$ — the full $d \times d$ gradient covariance. But this is $O(d^3)$. The diagonal $\text{diag}(\sum g_s^2)^{-1/2}$ is the affordable compromise.

Evolution of adaptive methods:

- **Adam /AdaGrad/ RMSProp:** Replace the sum with an exponential moving average and bias correction: $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$. This solves the vanishing step-size problem by “forgetting” past gradients (as in L-BFGS).
- **Shampoo:** Go back to the full matrix, but exploit the **matrix structure** of neural net weights — factor the preconditioner into row-space and column-space parts.
- **Muon:** Same matrix structure idea as Shampoo, but cleverly replaces the expensive matrix p -th root with a few Newton-Schulz iterations.

All are approximations to $H^{-1/2}$ at different points on the cost-vs-accuracy spectrum.

Muon: Motivation — Shampoo

For a weight matrix $W \in \mathbb{R}^{m \times n}$, let $G_t = \nabla_W L \in \mathbb{R}^{m \times n}$ be its gradient (a matrix, not a vector).

Shampoo (Gupta et al., 2018): The full AdaGrad preconditioner for a matrix parameter is $(\sum G_s G_s^T)^{-1/2}$, but that's $m^2 \times m^2$. Shampoo exploits matrix structure to factor it:

$$L_t = \varepsilon I + \sum_{s=1}^t G_s G_s^T \in \mathbb{R}^{m \times m}, \quad R_t = \varepsilon I + \sum_{s=1}^t G_s^T G_s \in \mathbb{R}^{n \times n}$$

L_t : correlations among **output** dimensions (rows of W).

R_t : correlations among **input** dimensions (columns of W).

The preconditioned gradient direction is $L_t^{-1/4} G_t R_t^{-1/4}$ and

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}.$$

This captures matrix structure, but computing $L^{-1/4}$ requires an eigendecomposition — expensive per step.

The **FULL** AdaGrad flattens the parameters as $\text{vec}(W) \in \mathbb{R}^{mn}$ and maintains

$$H_t = \epsilon I + \sum_{s=1}^t g_s g_s^\top \leq H_{t,r} := r\epsilon I + \sum_{s=1}^t g_s g_s^\top.$$

where r is the highest rank of all G_s . And (proof in arXiv:1802.09568)

$$\frac{1}{r}H_t \leq \frac{1}{r}H_{t,r} \leq L_t^{1/2} \otimes R_t^{1/2} \Rightarrow H_t^{-1/2} \geq (rL_t^{1/2} \otimes R_t^{1/2})^{-1/2} = \frac{1}{\sqrt{r}}L_t^{-1/4} \otimes R_t^{-1/4}.$$

The matrix $L_t^{-1/4} \otimes R_t^{-1/4}$ is still expensive to evaluate but (use $(A \otimes B)\text{vec}(C) = \text{vec}(ACB)$)

$$L_t^{-1/4} \otimes R_t^{-1/4} \text{vec}(G_t) = \text{vec}(L_t^{-1/4} G_t R_t^{-1/4}).$$

Muon: From Shampoo to Newton-Schulz

Key observation:

The roots $L_t^{-1/4}$ and $R_t^{-1/4}$ are not cheap at beginning stages.

If all G_t are orthogonal matrices, then L_t and R_t are just multiples of identity matrices. The update formula will simply $W_{t+1} = W_t - \eta G_t$.

Muon (Jordan, 2024): Instead of maintaining L_t, R_t , directly orthogonalize the momentum matrix via Newton-Schulz (finds a projection onto orthogonal matrices).

Muon: Algorithm

Muon Algorithm (per weight matrix W)

1. $M_t = \beta M_{t-1} + (1 - \beta) \nabla_W L$ (exponential moving average)
2. $O_t = \text{NS}(M_t)$ (orthogonalize via Newton-Schulz)
3. $W_t = W_t - \eta O_t$

Newton-Schulz iteration: A 5th-order polynomial on the singular values of X :

$$X_{k+1} = aX_k + (b \cdot A + c \cdot A^2)X_k, \quad A = X_k X_k^T$$

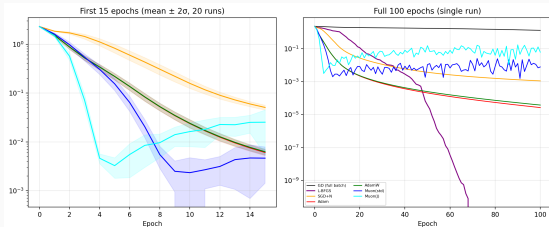
with $(a, b, c) = (3.4445, -4.7750, 2.0315)$. Pre-normalize $X_0 = X / \|X\|_F$, 5 steps.

The standard polar iteration $(1.5, -0.5, 0)$ is the 3rd-order method

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k X_k^T X_k.$$

Jordan's 5th-order tuning converges faster per step, designed for 'bf16' precision in large-scale training. (see Jordan's blog)

Optimizer Comparison



3-layer MLP, $d = 150$, $n = 3000$

GD	full batch, lr=0.1
SGD+N	lr=0.01, mom=0.9
L-BFGS	$m = 10$ histories
Adam	lr=0.001
AdamW	lr=0.001, wd=0.01
Muon(std)	std NS, 5 steps
Muon(J)	Jordan NS, 5 steps

Early training Jordan's tuned NS converges fastest early, ideal for early stopping. On deeper nets, Jordan NS can overshoot (as visible in the right panel: Muon(J) rises after epoch 30); the standard NS is more stable but slower.

Exercise: Stabilizing Muon

The problem: Jordan's tuned NS converges fast but is fragile — when M_t becomes ill-conditioned (one singular value dominates), NS rotates O_t sharply, causing the loss to spike.

Stabilization ideas:

Weight Decay: Like AdamW, see arXiv:2502.16982

Adaptive Muon: Like Adam, see arXiv:2507.11005

Conservative NS: The 3rd-order polar iteration or few NS steps $X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_kX_k^TX_k$ is inherently more stable than Jordan's 5th-order tuning. Is it possible to improve?

More: Can you make use of the spiky loss?

Modern Frontiers

Classical theory fails to explain modern deep learning:

- Overparameterized models *should* overfit — but they generalize better
- Learning rates often exceed the theoretical $2/L$ limit — but training still works
- Optimization algorithms select specific “structured” solutions without explicit regularization

These phenomena demand new theoretical frameworks.

The Edge of Stability (EoS)

Classical bound: $\eta < 2/L$ for stability.

Observation (Cohen et al., 2021): Deep learning optimizers operate with $\eta > 2/L$ yet converge!

Theorem (EoS Condition)

*During GD with step size η , the sharpest eigenvalue **self-stabilizes**:*

$$\lambda_{\max}(H(\mathbf{w}_t)) \approx \frac{2}{\eta}$$

See arXiv:2103.00065.

Along the sharpest direction v (eigenvector of H with eigenvalue λ), the loss is locally a parabola:

$$f(x) \approx \frac{1}{2}\lambda x^2, \quad x_{t+1} = (1 - \eta\lambda)x_t$$

Two regimes for the 1D GD dynamics $x_{t+1} = (1 - \eta\lambda)x_t$:

- $\eta\lambda < 2$: $|1 - \eta\lambda| < 1$, so $x_t \rightarrow 0$. As we descend toward the minimum, the landscape typically gets **sharper** (λ increases).
- $\eta\lambda > 2$: $|1 - \eta\lambda| > 1$, so x_t **diverges** — oscillates with growing amplitude. This pushes the iterate out of the sharp valley and into **flatter** regions (or just diverges).

In overparameterized models, infinitely many w achieve zero training error.

Which one does GD pick?

Theorem (Implicit Bias Towards Low Rank)

For matrix factorization $X = W_2 W_1$, GD from small initialization implicitly biases towards low-rank solutions (minimizing nuclear norm).

See NIPS 2017 and arXiv:1905.13655.

Implicit Bias: Example

There are infinitely many (w_1, w_2) give $w_1 w_2 = y$.

$\mathcal{L}(w_1, w_2) = \frac{1}{2}(w_1 w_2 - y)^2$. Let $\beta = w_1 w_2$.

Gradient flow:

$$\dot{w}_1 = -(w_1 w_2 - y)w_2 = -(\beta - y)w_2$$

$$\dot{w}_2 = -(w_1 w_2 - y)w_1 = -(\beta - y)w_1$$

Key observation: $w_1 \dot{w}_1 = w_2 \dot{w}_2$. Therefore:

$$\frac{d}{dt}(w_1^2 - w_2^2) = 2(w_1 \dot{w}_1 - w_2 \dot{w}_2) = 0$$

The difference $w_1^2 - w_2^2$ is **conserved**.

Question

What is the final state of w_1 and w_2 if they are initially around zero? Is it possible that the gradient flow does not converge to y ?

Implicit Bias: Example

Consequence for β : $\dot{\beta} = \dot{w}_1 w_2 + w_1 \dot{w}_2 = -2(\beta - y)\beta$. Compare to standard GD on β directly: $\dot{\beta} = -(\beta - y)$.

The extra β factor means: when β is small (early training), learning slows down, preferring balanced growth. This pushes toward low $\|(w_1, w_2)\|$ solutions.

Exercise

How about the general case of Deep Linear Network (DLN)? Implement the GD for DLN with different depths.

Grokking: Model achieves perfect training accuracy but zero test accuracy for many epochs, then suddenly generalizes — the solution “compresses” over time.

Interpretation: Optimization proceeds in stages:

1. Fit data with messy, high-complexity solution, needs regularization for sure.
2. Implicit regularization slowly compresses into a simpler, generalizable form (but depends on clean data and data distribution too).

See NIPS 2022 and [arXiv:2402.15555](https://arxiv.org/abs/2402.15555)

Essential References

1. Nesterov (1983). *A Method for Solving the Convex Programming Problem*.
2. Robbins & Monro (1951). *A Stochastic Approximation Method*.
3. Johnson & Zhang (2013). *Accelerating Stochastic Gradient Descent using Predictive Variance Reduction*.
4. Loshchilov & Hutter (2019). *Decoupled Weight Decay Regularization*.
5. Jordan (2024). *Muon: Momentum with Orthogonalization*.
6. Cohen et al. (2021). *Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability*.
7. Foret et al. (2021). *Sharpness-Aware Minimization*.